

AI College Recommendation System

PROJECT DOCUMENTATION

Project Title:

AI College Recommendation System

Intern's Name:

Archana K

Muhammed Shamaeel K M

Internship Period:

2 Months

Mentor / Supervisor Name:

Akhil Aashraf

Date of Submission:

30 / 05 / 2026

ABSTRACT

In today's rapidly growing educational landscape, students often face difficulties in finding suitable colleges, courses, fee structures, hostel facilities, placements, and admission details across different locations. Most educational websites provide static information and lack intelligent assistance for answering student queries based on actual college documents and brochures.

This project presents CampusIQ, an AI-powered College Discovery and Course Assistance platform that uses Retrieval-Augmented Generation (RAG) to provide accurate and intelligent college-related information. The system enables users to search colleges based on course and location preferences. Users can explore college details, course information, fee structures, eligibility criteria, placements, hostel facilities, and admission procedures.

The core intelligence of the platform is powered by RAG technology, where college brochure PDFs are processed into embeddings and stored in a vector database for semantic retrieval. Instead of relying solely on manually stored database records, the system retrieves relevant information directly from PDF documents and generates human-like responses using a Large Language Model running through Ollama. By combining modern AI technologies, semantic search, vector databases, and full-stack web development, CampusIQ delivers a smart, scalable, and user-friendly educational assistance platform.

OBJECTIVES

The main objective of CampusIQ is to provide an intelligent, automated platform that assists students in discovering colleges and obtaining accurate, contextual information through AI-driven techniques.

- To develop a centralized platform for searching colleges based on course and location
- To implement Retrieval-Augmented Generation (RAG) for intelligent college information retrieval
- To process and analyze college brochure PDFs as the primary knowledge source
- To generate semantic embeddings from educational documents
- To implement vector similarity search for accurate information retrieval
- To integrate an Ollama-based LLM for conversational question answering
- To provide detailed information about courses, fees, hostel facilities, placements, and admissions
- To develop a chatbot capable of answering user queries in natural language
- To build a scalable full-stack web application using Django and React
- To implement secure JWT-based authentication
- To create an efficient and user-friendly interface for students

PROBLEM STATEMENT

Students often struggle to gather accurate and complete information about colleges, courses, fee structures, placements, and hostel facilities. Existing educational portals generally provide static information and keyword-based search systems that lack contextual understanding and intelligent assistance.

Key challenges include:

- Difficulty in finding relevant colleges based on specific course and location preferences
- Lack of intelligent search systems capable of understanding natural language queries
- Educational information spread across multiple brochures and websites
- Manual searching through large PDF documents is time-consuming and inefficient
- Traditional systems cannot provide conversational assistance for user queries
- Absence of semantic search and contextual document retrieval
- Limited personalization and recommendation capabilities
- Static databases become outdated and difficult to maintain

SCOPE OF WORK

Will Cover

- College search based on course and location
- College listing and detail viewing system
- Processing of college brochure PDFs
- PDF text extraction and preprocessing
- Document chunking and embedding generation
- Vector database implementation using FAISS / ChromaDB
- Semantic similarity search
- RAG pipeline implementation
- Ollama LLM integration
- AI chatbot for user queries
- Course, fee, hostel, and placement information retrieval
- JWT authentication system
- REST API development
- Responsive frontend using React
- Chat history management
- Metadata filtering for colleges and courses

Will Not Cover

- Real-time admission application processing
- Online fee payment systems
- Official government admission verification
- Multi-language support (future scope)
- Voice assistant integration (future scope)
- Real-time university API integration
- Mobile application development

LITERATURE REVIEW

Existing educational platforms and college search systems primarily rely on manually entered database records and keyword-based filtering mechanisms. These systems frequently fail to provide contextual understanding and intelligent assistance for student queries.

Recent advancements in Artificial Intelligence, Natural Language Processing, and Large Language Models have enabled the development of intelligent information retrieval systems. Retrieval-Augmented Generation (RAG) combines semantic search with generative AI to retrieve relevant information from documents and generate accurate, contextual responses.

Vector databases such as FAISS and ChromaDB allow efficient storage and retrieval of semantic embeddings generated from textual data. Embedding models convert textual content into high-dimensional vectors that capture semantic meaning, enabling similarity-based search rather than simple keyword matching.

Modern LLM frameworks integrated with local AI runtimes such as Ollama provide efficient and privacy-focused conversational AI capabilities, making them well-suited for educational assistance applications.

FEATURES

1. College Discovery Features

- Course-based college search
- Location-based filtering
- College listing system
- Detailed college information pages
- Course detail viewing
- Fee structure retrieval
- Hostel and placement information

2. RAG & AI Features

- PDF document processing and text extraction
- Document chunking and embedding generation
- Semantic vector search
- Context retrieval system
- Retrieval-Augmented Generation (RAG) pipeline
- Metadata filtering
- Intelligent contextual answering

3. Chatbot Features

- AI-powered conversational chatbot
- Natural language query handling
- General-purpose student assistance
- College-related question answering
- Context-aware responses
- Chat history management

4. Platform Features

- JWT authentication and user registration
- REST API integration
- Responsive React-based UI
- Secure backend architecture
- Scalable and modular design

METHODOLOGY

The system follows a structured, modular workflow that progresses from user input through intelligent retrieval to AI-generated response delivery.

System Workflow:

User Query → Frontend Interface → Django REST API → RAG Pipeline → Embedding Search → Vector Database Retrieval → Ollama LLM → AI Response Generation → Frontend Display

RAG Workflow:

College PDF → Text Extraction → Document Chunking → Embedding Generation → Vector Database Storage

User Question → Query Embedding → Similarity Search → Relevant Chunk Retrieval → Ollama Prompt Generation → Final AI Response

The modular architecture ensures each component operates independently, enabling easy maintenance, updates, and parallel development across the system.

DATASET

Sources

- College brochure PDFs
- Fee structure documents
- Admission brochures
- Course information documents
- Placement reports

Preprocessing

- PDF text extraction
- Data cleaning and text normalization
- Chunk generation with overlap
- Metadata tagging per college and course
- Embedding creation and vector indexing

TECH STACK

Backend

- Python
- Django
- Django REST Framework

Frontend

- JavaScript
- React

Database

- PostgreSQL

AI / NLP

- LangChain
- SpaCy
- NLTK
- Sentence Transformers

LLM

- Ollama

RAG Components

- FAISS / ChromaDB (Vector Database)
- HuggingFace Embeddings
- Recursive Text Splitter

Authentication

- JWT Authentication

APIs

- REST API

TASK DIVISION

Intern 1 – Backend & AI Development

- Develop backend using Django and Django REST Framework
- Design APIs for colleges, chatbot, and search
- Implement RAG pipeline
- Develop PDF processing modules
- Implement embedding generation system
- Build vector database integration
- Integrate Ollama LLM
- Develop semantic retrieval system
- Manage PostgreSQL database
- Implement JWT authentication

Intern 2 – Frontend Development

- Develop frontend using React
- Design responsive UI pages
- Implement chatbot interface
- Build college listing and filtering pages
- Integrate frontend with backend APIs
- Develop authentication pages
- Implement search and navigation features
- Handle frontend state management
- Improve user experience and responsiveness

TIMELINE

Week 1

- Requirement analysis and architecture design
- Frontend and backend project setup
- Database configuration and JWT authentication setup
- PDF collection, text extraction, and preprocessing
- Document chunking and embedding generation
- Vector database setup
- RAG pipeline implementation
- Ollama LLM integration

Week 2

- College search system development
- AI chatbot development
- REST API integration
- Frontend UI development
- Semantic retrieval optimization
- Testing and debugging
- Performance optimization
- Final documentation and deployment preparation

EXPECTED DELIVERABLES

- Complete backend source code
- Complete frontend source code
- Functional RAG pipeline
- PDF processing modules
- Vector database implementation
- Ollama chatbot integration
- College search system
- Chatbot interface
- JWT authentication system
- REST API documentation
- Final project documentation
- Demo presentation

RISKS AND CHALLENGES

Identified risks:

- Extracting clean text from complex or scanned PDF brochures
- Maintaining accurate semantic retrieval across large document sets
- Managing embedding quality and retrieval precision
- Handling large document collections efficiently
- Reducing irrelevant AI responses (hallucinations)
- Optimizing vector search performance
- Integrating local LLM models efficiently
- Maintaining scalable backend performance

Mitigation Strategies:

- Use advanced PDF parsing and preprocessing techniques
- Implement metadata filtering for targeted retrieval
- Use high-quality, well-validated embedding models
- Apply chunk overlap techniques for better context coverage
- Optimize vector indexing for fast similarity search
- Use prompt engineering for accurate and grounded LLM responses
- Conduct regular testing and evaluation across scan scenarios
- Adopt modular architecture for scalability and easy updates

REFERENCES

- Django REST Framework – <https://www.django-rest-framework.org>
- React Documentation – <https://react.dev>
- LangChain Documentation – <https://docs.langchain.com>
- Ollama Documentation – <https://ollama.com/docs>
- FAISS Documentation – <https://faiss.ai>
- HuggingFace Sentence Transformers – <https://www.sbert.net>
- PostgreSQL Documentation – <https://www.postgresql.org/docs>
- SpaCy Documentation – <https://spacy.io>
- NLTK Documentation – <https://www.nltk.org>